Penjelasan kode praktik Pertemuan 11

Pegangan mengajar VB.NET Windows Forms: **Asisten Presentasi**.

Dokumen ini membahas setiap deklarasi API, fungsi bantu, variabel, dan event pada contoh praktik. Setiap bagian berisi kode dari proyek, penjelasan per baris atau kelompok baris, serta contoh kalimat untuk disampaikan kepada mahasiswa. Listing disinkronkan oleh `alat/dokumen_pertemuan11.py`.

Buka solution berdampingan dengan dokumen ini. PDF khusus penjelasan kode dapat digunakan ketika praktik; seluruh pembahasan juga disertakan dalam PDF naskah mengajar lengkap.

Urutan membuka kode di kelas

Kebutuhan penjelasan	Bagian
Bagaimana program dimulai dan kontrol dibuat	1–3
Bagaimana VB.NET mengenali fungsi Windows	4–6
Bagaimana waktu dihitung dan disajikan	7–9, 15–20
Bagaimana shortcut global bekerja	11–14
Bagaimana jendela diatur	21–22
Bagaimana menelusuri nilai dengan contoh angka	23

Tunjukkan hasil aplikasi lebih dahulu. Saat membaca kode, jelaskan kebutuhan dan inputnya, ikuti perubahan nilai, kemudian buktikan hasil pada aplikasi. Jangan membaca seluruh listing sekaligus tanpa demonstrasi.

1. Titik awal program: Program.vb

```
Option Strict On

Friend Module Program
    <STAThread()>
    Friend Sub Main()
        Application.SetHighDpiMode(HighDpiMode.SystemAware)
        Application.EnableVisualStyles()
        Application.SetCompatibleTextRenderingDefault(False)
        Application.Run(New FrmDemoApi())
    End Sub
End Module
```

Kode	Penjelasan untuk pengajar
`Option Strict On`	Meminta pemeriksaan tipe yang ketat saat kompilasi. Konversi yang mempersempit tipe perlu dinyatakan dengan jelas.
`Friend Module Program`	Menampung titik masuk program. `Friend` membatasi akses pada assembly yang sama. Anggota Module dapat dipakai tanpa membuat objek Module.
`<STAThread()>`	Mengatur model apartemen COM pada thread utama, sesuai kebutuhan berbagai layanan desktop Windows. Ini bukan perintah untuk membuat thread tambahan.
`Friend Sub Main()`	Prosedur yang dijalankan sebagai titik masuk. Pada `.vbproj`, `StartupObject` diatur ke `Sub Main`.
`SetHighDpiMode(SystemAware)`	Memilih perilaku DPI sebelum form dibuat. Ini membantu mengikuti scaling sistem; pengujian pada perangkat tujuan tetap diperlukan.
`EnableVisualStyles()`	Mengaktifkan tampilan kontrol sesuai visual style Windows.
`SetCompatibleTextRenderingDefault(False)`	Memilih mekanisme render teks standar bagi kontrol yang mendukung pengaturan tersebut.
`Application.Run(New FrmDemoApi())`	Membuat form utama dan menjalankan message loop. Loop memungkinkan klik, gambar ulang, Timer, dan pesan Windows diproses.
`End Sub`, `End Module`	Menutup blok prosedur dan Module.

Kalimat mengajar:

> Program dimulai dari Main. Kita menyiapkan perilaku tampilan terlebih dahulu, lalu membuat form utama. Application.Run menjaga aplikasi tetap memproses pesan selama form utama masih berjalan. Karena ada alur pesan inilah klik tombol dan shortcut dapat ditangani.

Pembuktian: letakkan breakpoint pada `Application.Run`, jalankan proyek, lalu lanjutkan. Form muncul setelah objek form dibuat.

2. Membaca kode Designer tanpa mencampurnya dengan logika

File `FrmDemoApi.Designer.vb` menyimpan pembentukan dan properti kontrol. Gunakan Designer Visual Studio untuk menata kontrol; kode penghitungan ditempatkan pada `FrmDemoApi.vb`.

Pola kode Designer	Arti dan alasan
`<DesignerGenerated()>`	Menandai bagian kelas yang dikelola perancang Visual Studio.
`Partial Class FrmDemoApi`	Menggabungkan bagian Designer dan bagian logika menjadi satu kelas saat kompilasi. Keduanya mengakses kontrol yang sama.
`Inherits System.Windows.Forms.Form`	Memberi kelas kemampuan dasar sebuah jendela WinForms.
`components` dan `New ComponentModel.Container()`	Menampung komponen yang perlu dikelola masa pakainya, termasuk Timer.
`Dispose(disposing As Boolean)`	Membersihkan komponen ketika form dibuang. `Finally` memastikan `MyBase.Dispose(disposing)` tetap dijalankan.
`InitializeComponent()`	Membentuk kontrol, mengatur properti, dan menyusun hubungan induk-anak. Prosedur ini dijalankan sebagai bagian inisialisasi form.
`pnlTimer = New Panel()`, `lblSisa = New Label()`	Membuat objek kontrol. `New` membuat instance, sedangkan variabelnya menyimpan rujukan ke instance tersebut.
`tmrHitung = New Timer(components)`	Membuat Timer WinForms nonvisual yang dimiliki container. Interval default Timer ini 100 ms; interval menentukan permintaan penyegaran UI.
`SuspendLayout()` / `ResumeLayout(False)`	Menunda lalu melanjutkan pemrosesan layout ketika banyak properti diatur. `False` pada ResumeLayout tidak langsung memaksa layout yang tertunda.
`BeginInit()` / `EndInit()`	Menandai awal dan akhir inisialisasi kontrol seperti NumericUpDown.
`Location = New Point(x, y)`	Menentukan posisi terhadap area induk kontrol. Label di dalam panel memakai koordinat panel tersebut.
`Size = New Size(lebar, tinggi)`	Menentukan ukuran kontrol. `ClientSize` pada form menyatakan area isi tanpa bingkai jendela.
`Controls.Add(kontrol)`	Memasukkan kontrol ke wadah. Label waktu dimasukkan ke `pnlTimer`, bukan sekadar digambar di atas panel.
`Anchor = Top Or Left Or Right`	Mempertahankan jarak terhadap sisi-sisi itu ketika induknya berubah ukuran. Label status memakai Bottom agar mengikuti bagian bawah form.

Pola kode Designer	Arti dan alasan
`Text`, `Font`, `ForeColor`, `BackColor`, `TextAlign`	Mengatur isi teks dan penyajiannya. `Color.FromArgb(r,g,b)` membentuk warna dari komponen merah, hijau, dan biru.
`Maximum`, `Value` pada NumericUpDown	Menentukan batas dan nilai awal. Menit maksimal 180; detik maksimal 59. Nilai minimum default adalah nol.
`New Decimal(New Integer() {5, 0, 0, 0})`	Bentuk kode yang dihasilkan Designer untuk nilai Decimal 5. Saat mengajar, ubah melalui properti Value agar mahasiswa fokus pada makna nilainya.
`Checked = True`	Memberi keadaan awal checkbox. Pengguna masih dapat mengubahnya saat aplikasi berjalan.
`TabIndex`, `AccessibleName`, `Cursor`	Mengatur urutan navigasi Tab, nama aksesibel, dan bentuk pointer.
`"&Jeda"`, `"M&enit"`	Tanda & menentukan mnemonic. Ini berbeda dari pendaftaran shortcut global Ctrl+Alt+F9.
`AutoScaleMode.Font`, `AutoScaleDimensions`	Menyimpan dasar scaling desain dan memilih scaling berdasarkan font.
`MaximizeBox = False`, `StartPosition = CenterScreen`	Menonaktifkan tombol maksimalkan dan menentukan posisi awal form.
`Name`, `Text` pada form	Name adalah identitas kontrol di kode; Text menjadi judul yang dilihat pengguna.
`Friend WithEvents btnMulai As Button`	Menyediakan variabel kontrol yang dapat dihubungkan ke handler menggunakan `Handles btnMulai.Click`.

Kalimat mengajar:

> Designer menjawab pertanyaan kontrol apa yang ada dan bagaimana tampilannya. File logika menjawab pertanyaan apa yang dilakukan ketika pengguna berinteraksi. Partial membuat kedua bagian itu tetap menjadi satu form. Kita bisa mengubah tampilan melalui Designer tanpa menempelkan perhitungan ke InitializeComponent.

3. Sintaks yang perlu dijelaskan sebelum membaca fungsi

```
Option Strict On
Imports System.Runtime.InteropServices

Public Module ModWindowsApi
```

`Imports System.Runtime.InteropServices` memungkinkan kita menulis `DllImport` dan `Marshal` tanpa mengulang nama namespace lengkap. Imports membantu penulisan nama; baris ini belum memanggil Windows API.

Sintaks	Cara membacanya dalam contoh
`Public Module ModWindowsApi`	Wadah deklarasi dan fungsi bersama yang digunakan form.
`Sub` / `Function ... As Tipe`	Sub melakukan pekerjaan tanpa nilai kembali; Function menghasilkan nilai bertipe tertentu.
`Private` / `Public`	Private membatasi anggota pada kelas atau Module pemiliknya; Public menyediakan akses sesuai jangkauan pemiliknya.
`Dim nama As Tipe`	Variabel lokal. Field `Private` di tingkat kelas bertahan selama instance form hidup.
`Me` / `MyBase`	Me menunjuk instance form ini; MyBase mengakses perilaku kelas dasarnya.
`Handles kontrol.Event`	Menghubungkan prosedur dengan event kontrol. `sender` adalah pemicu event; e membawa data event.
`If kondisi Then ...`, `Else`, `End If`	Memilih cabang berdasarkan kondisi Boolean.
`If(kondisi, nilaiBenar, nilaiSalah)`	Ekspresi yang menghasilkan salah satu dari dua nilai. Hanya cabang yang dipilih yang dievaluasi.
`Return` / `Return nilai`	Mengakhiri prosedur saat itu juga; Function dapat sekaligus mengembalikan hasil.
`Not` / `AndAlso`	Not membalik Boolean; AndAlso mengevaluasi syarat kanan hanya jika syarat kiri True.
`Or` pada konstanta UInteger	Menggabungkan bit flag. Pada contoh ini pemakaiannya berbeda dari pengujian kondisi Boolean.
`+=`	Menambahkan nilai ke isi variabel yang sudah ada.
`\` / `Mod`	Pembagian bilangan bulat dan sisa pembagian.

Sintaks	Cara membacanya dalam contoh
`&H40`, `UI`, `UL`, `D`	&H menyatakan heksadesimal; UI memberi tipe UInteger, UL memberi ULong, D memberi Decimal pada literal.
`"teks" & vbCrLf & "teks"`	Menggabungkan string dengan pergantian baris Windows.
`$"...{nilai}..."`	Interpolasi: memasukkan hasil ekspresi ke string. `:00` memakai sedikitnya dua digit.

Kalimat mengajar:

> Sebelum melihat API, pahami dulu tanda-tanda yang kita baca. Return menghentikan prosedur sekarang, sedangkan End Sub menutup definisi prosedurnya. Lalu perhatikan tipe angka: ULong kita gunakan untuk pencacah milidetik, dan IntPtr untuk handle. Tipe tersebut mengikuti data yang harus diwakili.

4. Konstanta API: nama untuk nilai yang disepakati

```
Public Const MB_ICONINFORMATION As UInteger = &H40UI
Public Const WM_HOTKEY As Integer = &H312
Public Const ID_MULAI As Integer = 1
Public Const ID_ULANGI As Integer = 2
Public Const MOD_CONTROL As UInteger = &H2UI
Public Const MOD_ALT As UInteger = &H1UI
Public Const MOD_NOREPEAT As UInteger = &H4000UI
Public Const VK_F9 As UInteger = &H78UI
Public Const VK_F10 As UInteger = &H79UI
Public Const SWP_NOSIZE As UInteger = &H1UI
Public Const SWP_NOMOVE As UInteger = &H2UI
Public Const SWP_NOACTIVATE As UInteger = &H10UI
Public ReadOnly HWND_TOPMOST As New IntPtr(-1)
Public ReadOnly HWND_NOTOPMOST As New IntPtr(-2)
```

Konstanta	Makna ketika dipakai
`MB_ICONINFORMATION = &H40UI`	Memilih jenis bunyi informasi Windows. Nama ini tidak membuat kotak pesan.
`WM_HOTKEY = &H312`	Nomor pesan yang kita periksa ketika shortcut terdaftar ditekan.
`ID_MULAI = 1`, `ID_ULANGI = 2`	ID pilihan aplikasi untuk membedakan tindakan shortcut. Ini bukan kode tombol F9/F10.
`MOD_CONTROL = &H2UI`, `MOD_ALT = &H1UI`	Flag kombinasi Ctrl dan Alt.
`MOD_NOREPEAT = &H4000UI`	Mencegah pesan berulang akibat autorepeat ketika tombol ditahan.
`VK_F9 = &H78UI`, `VK_F10 = &H79UI`	Kode virtual tombol F9 dan F10.
`SWP_NOSIZE = &H1UI`	Meminta SetWindowPos mempertahankan ukuran.
`SWP_NOMOVE = &H2UI`	Mempertahankan posisi.
`SWP_NOACTIVATE = &H10UI`	Meminta perubahan tanpa mengaktifkan jendela atau merebut fokus.
`HWND_TOPMOST = New IntPtr(-1)`	Nilai khusus parameter urutan jendela untuk masuk kelompok topmost.
`HWND_NOTOPMOST = New IntPtr(-2)`	Nilai khusus untuk keluar dari kelompok topmost.

`Const` menyimpan nilai tetap yang diketahui saat kompilasi. `ReadOnly` dipakai untuk kedua IntPtr karena objek nilai tersebut dibentuk saat inisialisasi, lalu field tidak ditugasi ulang dari alur biasa. Nilai -1 dan -2 di sini merupakan penanda khusus API, bukan handle jendela lain yang kita buat.

Kalimat mengajar:

> Angka ini berasal dari kontrak Windows atau pilihan ID aplikasi. Kita memberinya nama supaya pembaca tahu tujuan angka tersebut. Menggabungkan Ctrl dan Alt memakai Or berarti mengaktifkan dua bit pilihan sekaligus. Mengganti nilainya sembarangan dapat mengubah permintaan yang diterima Windows.

5. Declare: GetTickCount64 dan MessageBeep

```
Public Declare Function GetTickCount64 Lib "kernel32.dll" () As ULong

Public Declare Function MessageBeep Lib "user32.dll" (
    uType As UInteger) As Boolean
```

Bagian	Penjelasan
`Declare Function`	Menyatakan fungsi yang implementasinya ada di pustaka eksternal. Kita tidak menulis tubuh fungsi tersebut.
`GetTickCount64`	Nama fungsi Windows yang menghasilkan pencacah waktu sejak sistem mulai.
`Lib "kernel32.dll"`	Nama pustaka yang menyediakan fungsi. DLL ini bagian Windows, bukan berkas yang perlu disalin dari internet.
`()`	GetTickCount64 tidak memerlukan argumen.
`As ULong`	Menampung hasil unsigned 64 bit dalam satuan milidetik. Nilai ini bukan jam kalender.
`MessageBeep Lib "user32.dll"`	Deklarasi fungsi permintaan bunyi sistem.
`uType As UInteger`	Parameter jenis bunyi; pemanggil mengisinya dengan `MB_ICONINFORMATION`.
`As Boolean`	Hasil permintaan dipetakan dari BOOL native menjadi Boolean .NET. True menyatakan keberhasilan menurut kontrak API.

GetTickCount64 menghasilkan angka yang terus bertambah. Timer presentasi menyimpan satu pembacaan sebagai titik awal, lalu mengurangkannya dari pembacaan berikutnya. Resolusinya mengikuti timer sistem; jangan menyebutnya pengukur presisi untuk eksperimen submilidetik. Kontrak GetTickCount64.

Kalimat mengajar:

> Deklarasi ini baru memberi tahu VB.NET cara menemukan dan memanggil fungsi. Windows belum berbunyi hanya karena deklarasinya ditulis. Bunyi diminta ketika alur menjalankan MessageBeep dengan jenis bunyi tertentu. Begitu juga GetTickCount64: angkanya baru kita peroleh saat fungsi dipanggil.

6. DllImport: tiga kontrak yang memakai handle

```
<DllImport("user32.dll", ExactSpelling:=True, SetLastError:=True)>
Public Function SetWindowPos(hWnd As IntPtr, hWndInsertAfter As IntPtr,
    x As Integer, y As Integer, cx As Integer, cy As Integer,
    uFlags As UInteger) As Boolean
End Function
```

```
<DllImport("user32.dll", ExactSpelling:=True, SetLastError:=True)>
Public Function RegisterHotKey(hWnd As IntPtr, id As Integer,
    fsModifiers As UInteger, vk As UInteger) As Boolean
End Function
```

```
<DllImport("user32.dll", ExactSpelling:=True, SetLastError:=True)>
Public Function UnregisterHotKey(hWnd As IntPtr, id As Integer) As Boolean
End Function
```

`<DllImport(...)>` adalah atribut deklarasi P/Invoke. `ExactSpelling:=True` meminta pencarian nama fungsi persis seperti yang dinyatakan. `SetLastError:=True` meminta runtime menyimpan informasi last-error dari pemanggilan native agar dapat dibaca setelahnya. Atribut ini tidak menangani kegagalan atau menampilkan pesan sendiri. Penjelasan SetLastError.

Tubuh kosong sebelum `End Function` pada ketiga deklarasi ini bukan fungsi bantuan yang lupa diisi. Implementasinya berasal dari DLL yang ditentukan oleh atribut.

Fungsi dan parameter	Isi yang dikirim aplikasi
`SetWindowPos.hWnd`	`Me.Handle`, identitas jendela yang akan diatur.
`hWndInsertAfter`	Penanda TOPMOST atau NOTOPMOST, sesuai keadaan checkbox.
`x, y`	Posisi baru jika perpindahan tidak dinonaktifkan oleh flag.
`cx, cy`	Lebar dan tinggi baru jika perubahan ukuran tidak dinonaktifkan.
`uFlags`	Pilihan yang digabung dengan Or; contoh mempertahankan posisi, ukuran, dan fokus.
`RegisterHotKey.hWnd`	Handle penerima pesan shortcut.
`id`	ID tindakan yang nantinya dibaca dari pesan WM_HOTKEY.
`fsModifiers`	Gabungan Ctrl, Alt, dan NoRepeat.
`vk`	Kode tombol F9 atau F10.

Fungsi dan parameter	Isi yang dikirim aplikasi
`UnregisterHotKey.hWnd, id`	Pasangan handle dan ID yang pendaftarannya ingin dilepas.

`IntPtr` mengikuti ukuran pointer proses, sedangkan Integer tetap 32 bit. Parameter native HWND dipetakan ke IntPtr; parameter `int` ke Integer; UINT ke UInteger. Ketiga fungsi menghasilkan Boolean keberhasilan menurut kontraknya masing-masing. SetWindowPos, RegisterHotKey, UnregisterHotKey.

Kalimat mengajar:

> Perhatikan hubungan deklarasi dan pemanggilan nanti. Deklarasi menyebut tipe yang dapat diterima, sedangkan pemanggilan memberikan nilai untuk jendela ini. Jangan mengganti IntPtr menjadi Integer karena terlihat lebih sederhana. Kita harus menjaga representasi handle sesuai kontrak native.

7. HitungSisa: mencegah pengurangan melewati nol

```
Public Function HitungSisa(durasi As ULong, terpakai As ULong) As ULong
    If terpakai >= durasi Then Return 0UL
    Return durasi - terpakai
End Function
```

Baris	Penjelasan
`durasi As ULong, terpakai As ULong`	Dua input dalam satuan milidetik. Nama terpakai di sini adalah parameter lokal fungsi.
`If terpakai >= durasi Then Return 0UL`	Mengembalikan nol ketika durasi sudah tercapai atau terlampaui.
`Return durasi - terpakai`	Menghitung sisa hanya ketika pengurang masih lebih kecil dari durasi.

ULong tidak menyimpan bilangan negatif. Karena itu, pengujian batas dilakukan sebelum pengurangan. Contoh `HitungSisa(5000, 1200)` menghasilkan 3800; `HitungSisa(5000, 5200)` menghasilkan 0. Fungsi ini logika .NET biasa, bukan pemanggilan DLL.

Kalimat mengajar:

> Sisa waktu tidak boleh menjadi negatif. Kita memeriksa batas sebelum mengurangkan karena tipe yang dipakai unsigned. Meskipun pembaruan datang terlambat sampai waktu terpakai melewati target, fungsi tetap mengembalikan nol.

8. FormatSisa: dari milidetik menjadi menit dan detik

```
Public Function FormatSisa(milidetik As ULong) As String
    Dim detik As ULong = milidetik \ 1000UL
    If milidetik Mod 1000UL > 0UL Then detik += 1UL
    Return $"{detik \ 60UL:00}:{detik Mod 60UL:00}"
End Function
```

Baris	Penjelasan
`milidetik \ 1000UL`	Mengambil jumlah detik utuh dengan pembagian integer.
`milidetik Mod 1000UL > 0UL`	Memeriksa apakah masih ada pecahan detik.
`detik += 1UL`	Membulatkan ke atas jika masih ada pecahan. Sisa 1 ms tetap ditampilkan sebagai 1 detik.
`detik \ 60UL`	Menentukan bagian menit.
`detik Mod 60UL`	Menentukan bagian detik setelah menit dipisahkan.
`:00`	Memberi sedikitnya dua digit, sehingga 5 menjadi 05. Angka menit di atas 99 tetap ditampilkan lengkap.

Input	Detik setelah pembulatan	Tampilan
0 ms	0	00:00
1 ms	1	00:01
1000 ms	1	00:01
1001 ms	2	00:02
61001 ms	62	01:02

Kalimat mengajar:

> Fungsi ini hanya mengubah cara penyajian. Waktu yang dihitung tetap dalam milidetik. Kita membulatkan ke atas supaya layar belum menunjukkan nol selama masih ada sisa pecahan detik. Keputusan waktu habis tetap memakai nilai sisa asli.

9. Field form: menyimpan keadaan antar-event

```
Private durasi As ULong
Private terpakai As ULong
Private mulaiTick As ULong
Private berjalan As Boolean
Private selesai As Boolean
Private ringkas As Boolean
Private mengubahPin As Boolean
Private handleShortcut As IntPtr
Private shortcutMulai As Boolean
Private shortcutUlangi As Boolean
Private ukuranLengkap As Size
```

Field	Isi dan pemakaian
`durasi`	Target milidetik untuk sesi yang sedang disiapkan atau dijalankan.
`terpakai`	Akumulasi waktu dari segmen berjalan yang sudah ditutup oleh tindakan Jeda.
`mulaiTick`	Pembacaan pencacah ketika segmen berjalan terbaru dimulai.
`berjalan`	True ketika waktu sedang dihitung; False saat siap, dijeda, atau selesai.
`selesai`	True ketika sesi mencapai nol, agar tombol berikutnya dapat memulai sesi baru.
`ringkas`	Keadaan tampilan ringkas, terpisah dari keadaan waktu.
`mengubahPin`	Penjaga agar rollback checkbox tidak menjalankan permintaan pin secara berulang.
`handleShortcut`	Handle yang digunakan saat pendaftaran, kemudian dipakai kembali untuk cleanup.
`shortcutMulai`, `shortcutUlangi`	Hasil masing-masing pendaftaran, bukan asumsi bahwa semua shortcut aktif.
`ukuranLengkap`	Ukuran area isi form sebelum masuk mode ringkas, untuk dipulihkan nanti.

Field angka dimulai dari nol dan Boolean dari False. Nilainya bertahan ketika satu event selesai dan event lain diproses. Variabel lokal seperti `sisa` hanya diperlukan selama satu pemanggilan prosedur.

Kalimat mengajar:

> Klik Mulai dan klik Jeda terjadi pada waktu yang berbeda. Kita memerlukan field supaya keadaan sesi tetap tersimpan di antara dua klik itu. Terpakai menyimpan bagian yang sudah dijalani sebelum jeda; mulaiTick menyimpan awal segmen yang sedang berjalan.

10. Load: menyiapkan sesi awal

```
Private Sub FrmDemoApi_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    ukuranLengkap = Me.ClientSize
    UlangiTimer()
End Sub
```

`Handles MyBase.Load` menghubungkan prosedur ke event Load form. `ukuranLengkap = Me.ClientSize` menyimpan ukuran area isi yang aktual. `UlangiTimer()` kemudian membaca durasi dari kontrol dan menyiapkan keadaan siap.

Pendaftaran shortcut dipisahkan ke HandleCreated karena terikat pada keberadaan handle. Load bukan tempat untuk menganggap handle hanya dibuat sekali sepanjang umur objek form.

Kalimat mengajar:

> Saat form dibuka, kita simpan ukuran lengkap lalu siapkan timer. Daripada menyalin kode persiapan ke Load dan tombol Ulangi, keduanya memakai prosedur yang sama. Dengan begitu, keadaan awal dan keadaan setelah diulangi mengikuti aturan yang sama.

11. HandleCreated: mendaftarkan dua shortcut

```
Private Sub FrmDemoApi_HandleCreated(sender As Object, e As EventArgs) Handles MyBase.HandleCreated
    handleShortcut = Me.Handle
    Dim modifier As UInteger = MOD_CONTROL Or MOD_ALT Or MOD_NOREPEAT
    shortcutMulai = RegisterHotKey(handleShortcut, ID_MULAI, modifier, VK_F9)
    shortcutUlangi = RegisterHotKey(handleShortcut, ID_ULANGI, modifier, VK_F10)
    Dim mulai As String = If(shortcutMulai, "Ctrl+Alt+F9: mulai/jeda", "Shortcut mulai tidak
tersedia")
    Dim ulang As String = If(shortcutUlangi, "Ctrl+Alt+F10: ulangi", "Shortcut ulangi tidak
tersedia")
    lblShortcut.Text = mulai & vbCrLf & ulang & ". Tombol tetap dapat digunakan."
    If chkTeratas.Checked Then chkTeratas_CheckedChanged(Me, EventArgs.Empty)
End Sub
```

Baris atau kelompok	Penjelasan
`handleShortcut = Me.Handle`	Menyimpan identitas jendela penerima pesan. Handle yang sama dipakai untuk melepaskan pendaftaran.
`MOD_CONTROL Or MOD_ALT Or MOD_NOREPEAT`	Menggabungkan tiga bit pilihan menjadi satu nilai UInteger.
`RegisterHotKey(...ID_MULAI...VK_F9)`	Meminta kombinasi Ctrl+Alt+F9 untuk tindakan mulai/jeda.
`RegisterHotKey(...ID_ULANGI...VK_F10)`	Meminta Ctrl+Alt+F10 untuk tindakan ulangi.
`shortcutMulai = ..., shortcutUlangi = ...`	Menyimpan hasil masing-masing permintaan. Satu dapat berhasil ketika yang lain gagal.
`Dim mulai As String = If(...)`	Memilih petunjuk aktif atau keterangan tidak tersedia sesuai hasil nyata.
`lblShortcut.Text = ...`	Menampilkan dua status dengan baris baru dan mengingatkan bahwa tombol masih dapat digunakan.
`If chkTeratas.Checked Then ...`	Menerapkan kembali pilihan topmost bila handle baru dibuat saat checkbox masih dicentang.

Kombinasi dapat gagal didaftarkan jika sudah digunakan. Menutup instance lain tidak otomatis mengulang pendaftaran pada instance yang masih terbuka; contoh mendaftar kembali ketika HandleCreated terjadi. Siklus HandleCreated.

Kalimat mengajar:

> Kita meminta Windows mengirim pesan ketika kombinasi tertentu ditekan. Hasil permintaannya disimpan satu per satu. Karena itu, petunjuk di layar mengikuti keberhasilan pendaftaran, dan tombol tetap bekerja ketika shortcut bentrok dengan aplikasi lain.

12. HandleDestroyed: melepas pendaftaran

```
Private Sub FrmDemoApi_HandleDestroyed(sender As Object, e As EventArgs) Handles
MyBase.HandleDestroyed
    If shortcutMulai Then UnregisterHotKey(handleShortcut, ID_MULAI)
    If shortcutUlangi Then UnregisterHotKey(handleShortcut, ID_ULANGI)
    shortcutMulai = False
    shortcutUlangi = False
    handleShortcut = IntPtr.Zero
End Sub
```

Baris	Penjelasan
`If shortcutMulai Then UnregisterHotKey(...)`	Meminta pelepasan hanya untuk shortcut mulai yang sebelumnya berhasil didaftarkan.
`If shortcutUlangi Then ...`	Melakukan hal yang sama secara terpisah untuk shortcut ulangi.
`shortcutMulai = False`, `shortcutUlangi = False`	Menghapus penanda pendaftaran aktif pada state aplikasi.
`handleShortcut = IntPtr.Zero`	Menandai bahwa aplikasi tidak lagi menyimpan handle pendaftaran. Ini bukan pemanggilan untuk menghancurkan jendela.

Contoh meminta cleanup tetapi tidak menampilkan diagnosis hasil UnregisterHotKey. Jangan menjelaskan baris ini seolah nilai kembali pelepasan sudah diperiksa. Tujuan pengajaran ialah memasangkan pendaftaran dengan pelepasan dan menangani handle yang dapat dibuat ulang.

Kalimat mengajar:

> Sumber daya yang didaftarkan perlu dilepas saat masa pemakaiannya berakhir. Kita memakai pasangan handle dan ID yang disimpan saat mendaftar. Sesudah itu, penanda lokal dikosongkan agar kode tidak menganggap pendaftaran lama masih berlaku.

13. WndProc: menerima pesan dari Windows

```
Protected Overrides Sub WndProc(ByRef m As Message)
    If m.Msg = WM_HOTKEY Then
        If m.WParam.ToInt32() = ID_MULAI AndAlso shortcutMulai Then MulaiAtauJeda()
        If m.WParam.ToInt32() = ID_ULANGI AndAlso shortcutUlangi Then UlangiTimer()
    End If
    MyBase.WndProc(m)
End Sub
```

Bagian	Penjelasan
`Protected Overrides Sub WndProc`	Mengganti penanganan prosedur pesan pada form turunan untuk menambahkan pemrosesan.
`ByRef m As Message`	Menerima struktur pesan melalui referensi, termasuk nomor pesan dan parameternya.
`m.Msg = WM_HOTKEY`	Menyaring pesan yang berkaitan dengan shortcut terdaftar.
`m.WParam.ToInt32() = ID_MULAI`	Membaca ID tindakan. WParam berisi ID shortcut, bukan kode F9 itu sendiri.
`AndAlso shortcutMulai`	Memastikan state mencatat pendaftaran mulai sebagai aktif sebelum memanggil tindakannya.
`MulaiAtauJeda()` / `UlangiTimer()`	Menjalankan logika yang juga digunakan tombol biasa.
`MyBase.WndProc(m)`	Meneruskan pesan ke pemrosesan bawaan form. Pesan lain tetap mendapatkan penanganan normal.

Alurnya: pengguna menekan kombinasi → Windows mengenali pendaftaran → WM_HOTKEY masuk → form memeriksa ID → prosedur tindakan dipanggil. Aplikasi ini tidak merekam seluruh tombol pengguna. Kontrak WM_HOTKEY.

Kalimat mengajar:

> Windows membawa kabar lewat pesan. Kita periksa jenis kabarnya, lalu ID tindakannya. Setelah tahu bahwa ini shortcut mulai, kita panggil prosedur yang sama seperti tombol Mulai. Cara ini menjaga agar pintu masuk yang berbeda menghasilkan aturan kerja yang sama.

14. Event tombol: pintu masuk menuju logika bersama

```
Private Sub btnMulai_Click(sender As Object, e As EventArgs) Handles btnMulai.Click
    MulaiAtauJeda()
End Sub
```

```
Private Sub btnUlangi_Click(sender As Object, e As EventArgs) Handles btnUlangi.Click
    UlangiTimer()
End Sub
```

Nama prosedur membantu pembaca, sedangkan `Handles btnMulai.Click` yang menghubungkannya dengan event. Prosedur pertama meneruskan pekerjaan ke `MulaiAtauJeda`; prosedur kedua ke `UlangiTimer`. Parameter `sender` dan `e` mengikuti kontrak event, meskipun contoh ini tidak memakai nilainya.

Kalimat mengajar:

> Handler klik sengaja pendek karena aturan sesi berada pada prosedur bersama. Jika nanti aturan mulai berubah, kita memperbaiki satu prosedur dan perilaku dari tombol maupun shortcut ikut berubah.

15. MulaiAtauJeda: memilih tindakan dari state

```
Private Sub MulaiAtauJeda()
    If berjalan Then
        PerbaruiHitungan()
        If selesai Then Return
        terpakai += GetTickCount64() - mulaiTick
        berjalan = False
        tmrHitung.Stop()
        btnMulai.Text = "&Lanjutkan"
        lblTahap.Text = "Dijeda"
        lblKeterangan.Text = "Waktu berhenti sampai Anda melanjutkan."
        lblStatus.Text = "Timer dijeda. Durasi dapat diubah setelah memilih Ulangi."
        Return
    End If
    If selesai Then UlangiTimer()
    If durasi = 0UL Then
        lblStatus.Text = "Isi durasi lebih dari 0 detik sebelum mulai."
        nudDetik.Focus()
        Return
    End If
    mulaiTick = GetTickCount64()
    berjalan = True
    nudMenit.Enabled = False
    nudDetik.Enabled = False
    btnMulai.Text = "&Jeda"
    lblTahap.Text = "Presentasi berlangsung"
    lblKeterangan.Text = "Sisa waktu presentasi Anda"
    lblStatus.Text = "Timer berjalan. Anda dapat berpindah ke aplikasi slide."
    tmrHitung.Start()
End Sub
```

Fungsi tombol bergantung pada keadaan sesi. Bacalah dari cabang teratas; jangan langsung melompat ke `tmrHitung.Start()`.

Urutan kode	Yang terjadi dan alasannya
`If berjalan Then`	Klik saat sesi berjalan diartikan sebagai permintaan Jeda.
`PerbaruiHitungan()`	Menyegarkan sisa berdasarkan pembacaan waktu saat tindakan dilakukan. Sesi mungkin sudah habis di antara dua Tick.
`If selesai Then Return`	Jika pembaruan tadi mengakhiri sesi, hentikan cabang jeda agar status selesai tidak ditimpa menjadi Dijeda.

Urutan kode	Yang terjadi dan alasannya
`terpakai += GetTickCount64() - mulaiTick`	Menambahkan lama segmen terbaru ke akumulasi.
`berjalan = False, tmrHitung.Stop()`	Menandai jeda dan menghentikan permintaan pembaruan UI.
Teks Lanjutkan, Dijeda, keterangan, status	Menyampaikan keadaan dan tindakan berikutnya. Isian durasi masih terkunci.
`Return` setelah cabang jeda	Mencegah alur jatuh ke bagian mulai pada klik yang sama.
`If selesai Then UlangiTimer()`	Ketika tombol Mulai lagi ditekan, siapkan sesi baru dengan durasi pilihan.
`If durasi = 0UL Then`	Menolak sesi berdurasi nol. Pesan diberikan dan fokus dipindah ke isian detik, lalu Return.
`mulaiTick = GetTickCount64()`	Menyimpan awal segmen baru, baik untuk mulai pertama kali maupun melanjutkan.
`berjalan = True`	Mengizinkan PerbaruiHitungan menghitung waktu.
`nudMenit.Enabled = False, nudDetik.Enabled = False`	Mengunci target agar tidak berganti ketika sesi berlangsung atau dijeda.
Teks Jeda dan status berjalan	Menjelaskan bahwa tombol sekarang akan menjeda dan pengguna boleh berpindah ke slide.
`tmrHitung.Start()`	Mengaktifkan event penyegaran. Sumber perhitungan tetap pencacah waktu.

Kalimat mengajar:

> Satu tombol menangani beberapa tindakan berdasarkan state. Ketika sedang berjalan, klik berarti jeda; ketika dijeda, klik berarti lanjut. Kita tidak menghapus terpakai saat melanjutkan. Kita hanya mengambil titik awal baru untuk segmen berikutnya, supaya waktu selama jeda tidak ikut dihitung.

Pembuktian: mulai 5 detik, jeda sekitar 2 detik, tunggu beberapa detik, kemudian lanjutkan. Sisa saat lanjut berasal dari durasi awal dikurangi waktu yang benar-benar berjalan.

16. UlangiTimer: membaca durasi dan membersihkan sesi

```
Private Sub UlangiTimer()
    tmrHitung.Stop()
    berjalan = False
    selesai = False
    terpakai = 0UL
    durasi = CULng(nudMenit.Value * 60D + nudDetik.Value) * 1000UL
    nudMenit.Enabled = True
    nudDetik.Enabled = True
    btnMulai.Text = "&Mulai presentasi"
    lblTahap.Text = "Timer presentasi"
    lblSisa.Text = FormatSisa(durasi)
    lblKeterangan.Text = "Atur durasi, lalu mulai ketika Anda siap."
    pnlTimer.BackColor = Color.FromArgb(23, 50, 77)
    lblStatus.Text = "Timer siap. Aktifkan Tetap di atas agar terlihat saat berganti aplikasi."
End Sub
```

Baris atau kelompok	Penjelasan
`tmrHitung.Stop()`	Menghentikan penyegaran sebelum state disiapkan ulang.
`berjalan = False`, `selesai = False`	Menetapkan keadaan siap, bukan dijeda dari sesi lama atau selesai.
`terpakai = 0UL`	Membuang akumulasi waktu sesi sebelumnya.
`nudMenit.Value * 60D + nudDetik.Value`	Mengubah menit ke detik, kemudian menambahkan komponen detik. Nilai NumericUpDown bertipe Decimal.
`CULng(...) * 1000UL`	Mengubah jumlah detik menjadi ULong lalu ke milidetik. Contoh 1 menit 30 detik menjadi 90000 ms.
`Enabled = True` pada kedua isian	Mengizinkan perubahan target setelah sesi diulangi.
Teks tombol, tahap, dan keterangan	Mengembalikan petunjuk keadaan siap.
`lblSisa.Text = FormatSisa(durasi)`	Menampilkan seluruh durasi pilihan karena waktu terpakai sudah nol.
`pnlTimer.BackColor = Color.FromArgb(23, 50, 77)`	Memulihkan warna panel setelah kemungkinan keadaan Waktu habis.
`lblStatus.Text = ...`	Memberi petunjuk memakai Tetap di atas saat berpindah aplikasi.

`mulaiTick` tidak wajib direset di sini karena perhitungan berhenti saat `berjalan = False`. Nilainya akan ditimpa ketika sesi berikutnya dimulai.

Kalimat mengajar:

> Ulangi mengembalikan sesi ke awal sesuai angka pada pengaturan. Perhatikan urutan satuannya: menit menjadi detik, detik menjadi milidetik. Kita mengosongkan waktu terpakai dan membuka kembali kedua isian, sehingga pengguna boleh mengganti target untuk sesi selanjutnya.

17. ValueChanged: memperbarui target ketika diizinkan

```
Private Sub Durasi_ValueChanged(sender As Object, e As EventArgs) Handles nudMenit.ValueChanged,
nudDetik.ValueChanged
    If Not berjalan AndAlso nudMenit.Enabled Then UlangiTimer()
End Sub
```

Satu prosedur menangani perubahan dua kontrol melalui daftar `Handles`. `Not berjalan` mensyaratkan timer sedang berhenti. `nudMenit.Enabled` membedakan keadaan siap dari jeda atau selesai: kedua isian masih dinonaktifkan pada sesi yang sudah dimulai. Jika kedua syarat terpenuhi, `UlangiTimer()` membaca ulang isian dan memperbarui tampilan.

Kalimat mengajar:

> Timer berhenti belum tentu berarti durasinya boleh diubah. Saat jeda, kita ingin menjaga target yang sama. Karena itu, kode memeriksa juga apakah pengaturan diizinkan. Setelah memilih Ulangi, isian aktif dan perubahan nilainya langsung terlihat pada sisa waktu.

18. Tick: meminta pembaruan tampilan

```
Private Sub tmrHitung_Tick(sender As Object, e As EventArgs) Handles tmrHitung.Tick
    PerbaruiHitungan()
End Sub
```

Event ini hanya memanggil `PerbaruiHitungan()`. Tidak ada `sisa -= 100` atau pengurangan satu detik per Tick. Event Timer WinForms diproses pada thread UI dan dapat terlambat ketika thread sibuk. Ketika mendapat giliran, pembaruan membaca pencacah yang baru.

Kalimat mengajar:

> Timer memberi giliran untuk memperbarui layar. Kita tidak menganggap setiap giliran selalu datang tepat pada waktunya. Jika satu pembaruan terlambat, perhitungan berikutnya tetap memakai selisih waktu yang baru, sehingga keterlambatan tidak diakumulasikan dengan menghitung jumlah Tick.

19. PerbaruiHitungan: menggabungkan segmen waktu

```
Private Sub PerbaruiHitungan()
    If Not berjalan Then Return
    Dim elapsed As ULong = terpakai + GetTickCount64() - mulaiTick
    Dim sisa As ULong = HitungSisa(durasi, elapsed)
    lblSisa.Text = FormatSisa(sisa)
    If sisa = 0UL Then WaktuHabis()
End Sub
```

Baris	Penjelasan
`If Not berjalan Then Return`	Mengabaikan pembaruan saat sesi tidak aktif.
`terpakai + GetTickCount64() - mulaiTick`	Menjumlahkan akumulasi sebelum jeda dengan lama segmen aktif sejak mulai atau lanjut.
`HitungSisa(durasi, elapsed)`	Mengurangi target dengan total waktu terpakai, dengan perlindungan batas nol.
`lblSisa.Text = FormatSisa(sisa)`	Mengubah nilai sisa menjadi teks menit:detik.
`If sisa = 0UL Then WaktuHabis()`	Mengakhiri sesi berdasarkan nilai milidetik, bukan berdasarkan string di Label.

Contoh: `durasi = 5000`, `terpakai = 2000`, `mulaiTick = 200000`, pencacah sekarang `201250`. Maka `elapsed = 3250`, `sisa = 1750`, dan tampilan `00:02`. Semua angka pencacah contoh ini adalah ilustrasi untuk penelusuran, bukan hasil pengukuran perangkat tertentu.

Kalimat mengajar:

> Waktu total berasal dari bagian sebelum jeda ditambah bagian yang sedang berjalan. Setelah itu, kita hitung sisa dan baru mengubahnya menjadi teks. Jadi, nilai perhitungan dan cara menampilkannya dipisahkan. Label bukan tempat menyimpan sumber kebenaran waktu.

20. WaktuHabis: mengakhiri sesi sebelum meminta bunyi

```
Private Sub WaktuHabis()
    tmrHitung.Stop()
    berjalan = False
    selesai = True
    lblTahap.Text = "Waktu habis"
    lblKeterangan.Text = "Akhiri presentasi dan masuk ke tanya jawab."
    pnlTimer.BackColor = Color.FromArgb(133, 52, 21)
    btnMulai.Text = "&Mulai lagi"
    lblStatus.Text = "Durasi selesai. Pilih Ulangi untuk mengatur sesi berikutnya."
    If chkBunyi.Checked Then
        If Not MessageBeep(MB_ICONINFORMATION) Then
            lblStatus.Text = "Waktu habis. Bunyi tidak dapat diputar; penanda visual tetap aktif."
        End If
    End If
End Sub
```

Baris atau kelompok	Penjelasan
`tmrHitung.Stop()`, `berjalan = False`	Menghentikan pembaruan rutin dan mencegah alur berjalan memproses akhir sesi lagi.
`selesai = True`	Menyimpan bahwa tombol berikutnya dapat memulai sesi baru.
Label tahap dan keterangan	Mengumumkan Waktu habis dan transisi ke tanya jawab.
`Color.FromArgb(133, 52, 21)`	Mengubah penanda visual panel. Informasi juga disampaikan lewat teks.
`btnMulai.Text = "&Mulai lagi"`	Mengganti petunjuk tindakan berikutnya.
`lblStatus.Text = ...`	Mengarahkan pengguna untuk memilih Ulangi jika ingin mengubah sesi.
`If chkBunyi.Checked Then`	Menghormati pilihan pengguna. Jika tidak dicentang, MessageBeep tidak dipanggil.
`If Not MessageBeep(MB_ICONINFORMATION) Then`	Meminta bunyi informasi dan memeriksa status. Cabang dijalankan ketika hasil False.
Pesan kegagalan bunyi	Memberi keterangan bahwa permintaan bunyi tidak berhasil; penanda visual tetap tersedia.

True dari MessageBeep bukan bukti bahwa manusia mendengar suara. Volume, perangkat, dan skema suara masih berpengaruh. Contoh memeriksa status tanpa mencoba membaca last-error melalui deklarasi Declare ini. Kontrak MessageBeep.

Kalimat mengajar:

> Kita akhiri state sesi lebih dahulu dan tampilkan hasilnya. Bunyi hanya tambahan sesuai pilihan pengguna. Kalau bunyi tidak terdengar, pengguna tetap mendapat tulisan Waktu habis. Keberhasilan permintaan API juga harus dibedakan dari hasil yang bisa didengar pada perangkat kelas.

21. CheckedChanged: mengatur posisi teratas dan rollback

```
Private Sub chkTeratas_CheckedChanged(sender As Object, e As EventArgs) Handles
chkTeratas.CheckedChanged
    If mengubahPin Then Return
    Dim posisi As IntPtr = If(chkTeratas.Checked, HWND_TOPMOST, HWND_NOTOPMOST)
    Dim flags As UInteger = SWP_NOMOVE Or SWP_NOSIZE Or SWP_NOACTIVATE
    If SetWindowPos(Me.Handle, posisi, 0, 0, 0, 0, flags) Then
        lblStatus.Text = If(chkTeratas.Checked, "Timer tetap di atas jendela biasa. Gunakan Mode
ringkas saat presentasi.", "Jendela kembali mengikuti urutan aplikasi biasa.")
    Else
        Dim kode As Integer = Marshal.GetLastPInvokeError()
        mengubahPin = True
        chkTeratas.Checked = Not chkTeratas.Checked
        mengubahPin = False
        lblStatus.Text = $"Posisi jendela tidak dapat diubah. Kode Windows: {kode}."
    End If
End Sub
```

Baris atau kelompok	Penjelasan
`If mengubahPin Then Return`	Mengabaikan event yang dipicu rollback internal checkbox.
`If(chkTeratas.Checked, HWND_TOPMOST, HWND_NOTOPMOST)`	Memilih permintaan masuk atau keluar dari kelompok topmost.
`SWP_NOMOVE Or SWP_NOSIZE Or SWP_NOACTIVATE`	Menggabungkan pilihan mempertahankan posisi, ukuran, dan fokus.
`SetWindowPos(Me.Handle, posisi, 0, 0, 0, 0, flags)`	Meminta Windows mengubah urutan jendela ini. Empat nol bukan permintaan menjadikannya berukuran nol karena posisi/ukuran diabaikan sesuai flag.
Cabang `Then`	Menampilkan status yang sesuai dengan pilihan pin yang berhasil diterapkan.
`Marshal.GetLastPInvokeError()`	Membaca kode error yang disimpan runtime setelah pemanggilan native gagal. Diletakkan sebelum pekerjaan lain pada cabang kegagalan.
`mengubahPin = True`	Mengaktifkan penjaga sebelum checkbox dikembalikan.
`chkTeratas.Checked = Not chkTeratas.Checked`	Membalik checkbox ke pilihan sebelumnya agar UI tidak mengklaim permintaan gagal sebagai keadaan aktif. Perubahan ini memicu CheckedChanged lagi.

Baris atau kelompok	Penjelasan
`mengubahPin = False`	Membuka kembali pemrosesan untuk tindakan pengguna berikutnya.
String berisi `{kode}`	Menampilkan informasi diagnosis yang diterima dari Windows.

Tanpa penjaga, perubahan checkbox di dalam handler dapat meminta perubahan native lagi. `SetLastError:=True` pada deklarasi mendukung pembacaan kode error di cabang ini. Topmost mengatur urutan jendela, bukan ukuran maksimumnya, dan tidak menjamin berada di atas seluruh jenis tampilan Windows. Pembacaan error P/Invoke.

Kalimat mengajar:

> Kita meminta timer tetap di atas jendela biasa tanpa memindahkan posisi atau merebut fokus. Hasil permintaannya diperiksa. Jika gagal, checkbox dikembalikan supaya tampilan tidak memberi informasi palsu. Penjaga mengubahPin mencegah pengembalian checkbox menjalankan permintaan yang sama secara berulang.

Pembuktian: buka Notepad, aktifkan Tetap di atas, lalu pindahkan fokus ke Notepad. Timer tetap terlihat pada jendela biasa, tetapi pengetikan tetap masuk ke Notepad.

22. Mode ringkas: perubahan tampilan dengan fasilitas .NET

```
Private Sub btnRingkas_Click(sender As Object, e As EventArgs) Handles btnRingkas.Click
    If Not ringkas Then ukuranLengkap = Me.ClientSize
    ringkas = Not ringkas
    pnlPengaturan.Visible = Not ringkas
    btnRingkas.Text = If(ringkas, "&Pengaturan", "Mode &ringkas")
    Dim bingkai As New Size(Me.Width - Me.ClientSize.Width, Me.Height - Me.ClientSize.Height)
    Dim tinggi As Integer = If(ringkas, pnlPengaturan.Top + lblStatus.Height + 24,
ukuranLengkap.Height)
    Me.MinimumSize = New Size(ukuranLengkap.Width + bingkai.Width, tinggi + bingkai.Height)
    Me.ClientSize = New Size(ukuranLengkap.Width, tinggi)
End Sub
```

Baris	Penjelasan
`If Not ringkas Then ukuranLengkap = Me.ClientSize`	Menyimpan ukuran lengkap tepat sebelum diperkecil. Ketika kembali, ukuran kecil tidak menimpa ukuran simpanan.
`ringkas = Not ringkas`	Membalik state tampilan pada setiap klik.
`pnlPengaturan.Visible = Not ringkas`	Menyembunyikan panel pada mode ringkas dan menampilkannya kembali pada mode lengkap.
`btnRingkas.Text = If(...)`	Menampilkan tindakan berikutnya: Pengaturan untuk kembali, Mode ringkas untuk memperkecil.
`New Size(Me.Width - Me.ClientSize.Width, ...)`	Mengukur selisih ukuran luar terhadap area isi, yaitu ruang bingkai dan judul jendela.
`Dim tinggi As Integer = If(...)`	Memilih tinggi ringkas berdasarkan letak panel dan tinggi status, atau tinggi lengkap yang disimpan. Angka 24 menyediakan ruang tambahan untuk pesan dan tepi bawah.
`Me.MinimumSize = ...`	Menetapkan batas ukuran luar yang sesuai dengan mode tujuan, termasuk bingkai.
`Me.ClientSize = ...`	Mengubah ukuran area isi. Lebar lengkap dipertahankan, tingginya mengikuti mode.

Bagian ini memakai properti WinForms; tidak memerlukan P/Invoke tambahan. Mode ringkas tidak mengubah durasi, state berjalan, atau pendaftaran shortcut. Ukuran dihitung dari ukuran aktual form, bukan hanya mengasumsikan ukuran bingkai dari satu konfigurasi layar.

Kalimat mengajar:

> Untuk menyederhanakan tampilan, fasilitas WinForms sudah mencukupi. Kita simpan ukuran lengkap, sembunyikan pengaturan, lalu ubah tinggi form. Perhatikan perbedaan ukuran luar dan ClientSize. Batas MinimumSize ikut disesuaikan agar tidak menahan form pada ukuran mode sebelumnya.

23. Menelusuri satu sesi dengan contoh angka

Gunakan contoh target **5 detik**. Nilai pencacah berikut sengaja dibuat sederhana untuk latihan membaca alur.

Tindakan	Perubahan data	Hasil yang dijelaskan
Ulangi	`durasi = 5000`, `terpakai = 0`, `berjalan = False`, `selesai = False`	Siap 00:05; isian aktif.
Mulai ketika pencacah 100000	`mulaiTick = 100000`, `berjalan = True`	Isian terkunci; Timer aktif.
Pembaruan pada 101200	`elapsed = 1200`, `sisa = 3800`	Tampilan 00:04.
Jeda sekitar 102000	`terpakai` menjadi sekitar 2000; `berjalan = False`	Tampilan sekitar 00:03; Timer berhenti. Dua pembacaan dalam cabang jeda dapat berselisih sedikit.
Menunggu ketika jeda	Tidak ada tambahan ke `terpakai`	Waktu menunggu tidak mengurangi sisa.
Lanjut pada 200000	`mulaiTick = 200000`, `terpakai` tetap sekitar 2000	Perhitungan segmen baru dimulai.
Pembaruan pada 201250	Total sekitar 3250 ms; sisa sekitar 1750 ms	Tampilan 00:02.
Segmen baru mencapai sekitar 3000 ms	Total mencapai target; HitungSisa menghasilkan nol	WaktuHabis menghentikan Timer dan memberi penanda.
Mulai lagi	UlangiTimer dipanggil, lalu segmen baru dimulai	Memakai durasi pilihan yang sama.
Ulangi untuk mengganti target	Isian dibuka kembali	Perubahan isian memicu ValueChanged.

Pertanyaan yang bisa diajukan sambil menunjuk kode

1. **Mengapa tidak memakai nilai GetTickCount64 langsung sebagai sisa?** Karena nilainya merupakan pencacah sejak sistem mulai. Aplikasi memerlukan selisih terhadap awal segmen.
2. **Mengapa jeda tidak menghapus terpakai?** Nilai itu diperlukan untuk menjaga kemajuan sesi ketika dilanjutkan.

3. **Mengapa 1750 ms tampil 00:02?** FormatSisa membulatkan ke atas; keputusan selesai memakai sisa milidetik.
4. **Mengapa ada Return setelah cabang jeda?** Supaya klik yang baru menjeda tidak langsung menjalankan bagian mulai.
5. **Apakah semua fungsi pada Module adalah API?** Tidak. HitungSisa dan FormatSisa merupakan fungsi bantuan yang kita tulis sendiri.
6. **Mengapa event tombol dan WndProc memanggil prosedur yang sama?** Agar validasi dan perubahan state konsisten dari kedua pintu masuk.
7. **Apakah satu False berarti seluruh aplikasi gagal?** Tidak. Arti hasil mengikuti fungsi; contoh tetap menyediakan tombol saat pendaftaran shortcut gagal dan penanda visual ketika bunyi gagal.

Kalimat mengajar:

> Sekarang ikuti datanya, jangan hanya nama fungsi. Kita tahu durasi targetnya, akumulasi sebelum jedanya, dan titik awal segmen terbaru. Dari tiga hal itu kita bisa menghitung total waktu dan sisa. Kalau hasil aplikasi berbeda dari dugaan, periksa perubahan nilai tersebut pada breakpoint sebelum mengganti kode.

Rujukan dan batas pembahasan

Listing berasal dari Program.vb, ModWindowsApi.vb, dan FrmDemoApi.vb. Bagian Designer dijelaskan per pola properti dari FrmDemoApi.Designer.vb; pengulangan properti setiap Label tidak dicetak ulang. Semua prosedur logika dan deklarasi native pada dua file utama tercakup dalam dokumen ini.

Dokumen ini menjelaskan praktik Asisten Presentasi. Tugas Panel Komputer Laboratorium mempunyai spesifikasi sendiri dan kunci jawabannya tidak disertakan di sini. Penjelasan kontrak API diperiksa pada dokumentasi Microsoft yang ditautkan di bagian terkait. Dokumen tidak mengklaim bahwa cabang kegagalan SetWindowPos atau hasil pendengaran bunyi sudah diuji ulang.